

The slide features a light gray background with several decorative elements: a large white circle in the top-left corner, a solid purple circle in the top-left area, a purple-to-white gradient bar in the top-right corner, a small purple circle in the bottom-right area, a large purple-to-white gradient circle in the bottom-right corner, and a purple-to-white gradient bar in the bottom-left corner.

PROG IV: **CRUD e Padrão REST**

Prof^a. Tainá Isabela

CRUD

CRUD é um acrônimo para **Create** (Criar), **Read** (Ler), **Update** (Atualizar) e **Delete** (Deletar) — as quatro operações fundamentais realizadas sobre dados em qualquer aplicação de software. Essas operações representam a base de como sistemas armazenam, acessam, manipulam e excluem informações de forma persistente em bancos de dados.

Independentemente da linguagem de programação, estrutura da aplicação ou tipo de banco de dados, essas quatro ações estão presentes de forma recorrente em praticamente todos os sistemas digitais modernos: desde um app de tarefas até uma aplicação bancária robusta.

CRUD

O conceito de CRUD não é apenas uma sigla técnica, mas sim um padrão universal de desenvolvimento de sistemas. Ele surgiu como forma de organizar logicamente as interações entre usuário e banco de dados. Na prática, toda ação que o usuário realiza em uma interface — como cadastrar, visualizar, editar ou excluir algo — corresponde diretamente a uma das operações CRUD.

CRUD

Por exemplo:

- Criar uma nova conta: Create
- Visualizar seus dados: Read
- Atualizar endereço: Update
- Deletar perfil: Delete

CRUD

CRUD é a espinha dorsal de sistemas de gerenciamento de conteúdo (CMS), ERPs, CRMs, lojas virtuais, redes sociais, APIs RESTful, apps mobile e qualquer aplicação que envolva dados estruturados. A adoção desse modelo garante organização, previsibilidade e escalabilidade no desenvolvimento.

CRUD

O funcionamento do CRUD pode ser descrito em uma tabela que relaciona as quatro operações às suas correspondências em HTTP, SQL e exemplos em sistemas reais:

Operação	Ação	Método HTTP	Comando SQL	Exemplo em sistema
Create	Criar um novo dado	POST	INSERT INTO	Cadastro de um novo produto
Read	Ler dados existentes	GET	SELECT	Listagem de clientes
Update	Atualizar dados existentes	PUT/PATCH	UPDATE	Alterar endereço do cliente
Delete	Remover dados existentes	DELETE	DELETE FROM	Excluir pedido cancelado

CRUD

Na arquitetura RESTful, cada uma dessas ações se alinha a um método HTTP, facilitando a organização de rotas e a integração entre front-end e back-end. No banco de dados, são instruções SQL que manipulam as informações de forma estruturada e segura.

CRUD

Considere um sistema simples de gerenciamento de usuários. Usando Node.js com Express, a estrutura de rotas pode ser representada assim:

// Create – Cadastrar novo usuário

```
app.post('/usuarios', (req, res) => {
```

// lógica para adicionar usuário

```
});
```

// Read – Listar usuários

```
app.get('/usuarios', (req, res) => {
```

// lógica para buscar usuários

```
});
```

CRUD

Considere um sistema simples de gerenciamento de usuários. Usando Node.js com Express, a estrutura de rotas pode ser representada assim:

// Update – Atualizar usuário existente

```
app.put('/usuarios/:id', (req, res) => {
```

// lógica para atualizar dados

```
});
```

// Delete – Remover usuário

```
app.delete('/usuarios/:id', (req, res) => {
```

// lógica para deletar usuário

```
});
```

CRUD

CRUD é um padrão que independe da tecnologia utilizada, o que significa que ele é amplamente aplicável. Veja algumas das tecnologias mais populares que utilizam essa estrutura:

- JavaScript: Express.js, Next.js, NestJS
- Python: Django, Flask, FastAPI
- Java: Spring Boot, Jakarta EE
- PHP: Laravel, Symfony
- Ruby: Ruby on Rails
- C#: ASP.NET Core
- NoSQL: MongoDB, Firebase, DynamoDB

Esses frameworks e bibliotecas oferecem ferramentas integradas para criação rápida de rotas, controllers e modelos de dados baseados em operações CRUD.

Vantagens de usar CRUD

Adotar a estrutura CRUD nos seus projetos de software oferece diversas vantagens, entre elas:

- **Organização do código:** separa cada operação em módulos claros e fáceis de manter.
- **Padronização:** torna o sistema mais intuitivo e consistente para desenvolvedores.
- **Escalabilidade:** facilita a evolução do sistema, adicionando novas tabelas ou entidades.
- **Integração com front-end:** APIs CRUD são facilmente consumidas por frameworks como React, Vue ou Angular.
- **Facilidade de teste:** cada operação pode ser testada individualmente.
- **Documentação simplificada:** ferramentas como Swagger automatizam a documentação de rotas CRUD.

Diferença entre CRUD e REST

Apesar de muitas vezes aparecem juntos, CRUD e REST não são a mesma coisa. Observe a diferença entre os dois conceitos:

CRUD	REST
Operações básicas sobre dados	Estilo arquitetural para APIs
C -> Create, R -> Read, etc.	GET, POST, PUT, DELETE
Conceito lógico de manipulação	Estrutura de comunicação via HTTP

Enquanto o CRUD define o que será feito com os dados, o REST define como essas ações são organizadas e expostas em uma API. Na prática, uma API RESTful quase sempre implementa um CRUD por trás de suas rotas.

Boas práticas para implementar CRUD

Mesmo sendo simples, um CRUD mal feito pode gerar sistemas difíceis de escalar. Aqui estão boas práticas essenciais:

- Use convenções claras de nomeação (ex: /usuarios em vez de /cadastro-usuario)
- Adote versionamento de API (ex: /api/v1/usuarios)
- Valide entradas e saídas de dados com bibliotecas como Joi, Zod ou pydantic
- Implemente autenticação e autorização (ex: JWT, OAuth)
- Documente todas as rotas com Swagger/OpenAPI
- Separe responsabilidades em camadas (Model, Controller, Service)
- Utilize testes automatizados para cada rota (unit e integration)

Princípios da arquitetura REST

No nível mais básico, uma API é um mecanismo que permite que uma aplicação ou serviço acesse um recurso presente em outra aplicação, serviço ou banco de dados. A aplicação ou serviço que acessa os recursos é o cliente e a aplicação ou serviço que contém o recurso é o servidor.

Algumas APIs, como SOAP ou XML-RPC, impõem um framework rígido para os desenvolvedores. Mas os desenvolvedores podem criar APIs REST usando praticamente qualquer linguagem de programação e vários formatos de dados. A única exigência é que sigam os seis princípios de design REST descritos a seguir, também chamados de restrições arquitetônicas.

Princípios da arquitetura REST

Interface uniforme

Todas as solicitações de API pelo mesmo recurso devem ter a mesma aparência, independentemente da origem. A API REST deve garantir que o mesmo dado, como o nome ou endereço de e-mail de um usuário, pertença a somente um identificador de recurso uniforme (URI). Os recursos não devem ser muito grandes, mas devem conter todas as informações que o cliente possa precisar.

Princípios da arquitetura REST

Desacoplamento cliente-servidor

No design da API REST, as aplicações do cliente e do servidor devem ser independentes uma da outra. A única informação que a aplicação do cliente deve conhecer é o URI do recurso solicitado; ela não pode interagir com a aplicação do servidor de nenhum outro jeito. Da mesma forma, uma aplicação de servidor não deve modificar a aplicação cliente, exceto por meio do envio dos dados solicitados via HTTP.

Princípios da arquitetura REST

Sem estado

As APIs REST são sem estado, ou seja, cada solicitação precisa incluir todas as informações necessárias para seu processamento. Em outras palavras, as APIs REST não requerem quaisquer sessões do lado do servidor. Aplicações do servidor não estão autorizadas a armazenar quaisquer dados relacionados a uma solicitação do cliente.

Princípios da arquitetura REST

Possibilidade de armazenamento em cache

Quando possível, os recursos devem ser armazenados em cache no lado do cliente ou do servidor. As respostas do servidor também precisam conter informações sobre se o cache é permitido para o recurso entregue. O objetivo é melhorar o desempenho no lado do cliente e, ao mesmo tempo, aumentar a escalabilidade no lado do servidor.

Princípios da arquitetura REST

Arquitetura de sistema em camadas

Nas APIs REST, as chamadas e respostas passam por diferentes camadas. Como regra de ouro, não presume que as aplicações cliente e servidor se conectem diretamente entre si. Pode haver vários intermediários diferentes no circuito de comunicação. As APIs REST precisam ser projetadas de modo que nem o cliente nem o servidor possam dizer se estão se comunicando com a aplicação final ou um intermediário.

Princípios da arquitetura REST

Código sob demanda (opcional)

As APIs REST geralmente enviam recursos estáticos, mas, em certos casos, as respostas também podem conter código executável. Nesses casos, o código deve ser executado apenas sob demanda.

Projeto Nest.JS – Continuando

```
import { Module } from '@nestjs/common';
import { AppController } from './app.controller';
import { AppService } from './app.service';
import { UsersModule } from './users/users.module';

@Module({
  imports: [UsersModule],
  controllers: [AppController],
  providers: [AppService],
})
export class AppModule {}
```

Primeiro, verificar se a importação do módulo de pessoas foi importado para nosso projeto principal, dentro de imports.

Definindo entidade user (ou pessoa)

O arquivo **user.entity.ts** ficará como abaixo:

```
export class User {  
  id: number;  
  nome: string;  
  idade: number;  
}
```

Configurando banco de dados

Temos a nossa entidade já com suas devidas propriedades, mas precisamos ter um banco de dados para salvar e manipular essas informações, para isso, vamos utilizar o TypeORM com o SQLite para fins didáticos.

Para instalar o TypeORM e o driver de conexão do SQLite vamos utilizar o npm, com o seguinte comando:

```
npm i @nestjs/typeorm typeorm sqlite3
```

Configurando banco de dados

Após a instalação, vamos criar o arquivo de configuração do banco de dados no diretório `/src` e vamos chama-lo de **ormconfig.ts**:

```
import { DataSourceOptions } from 'typeorm';

export const config: DataSourceOptions = {
  type: 'sqlite',
  database: 'db/sql',
  synchronize: true, // Obs: use synchronize: true somente em desenvolvimento.
  entities: ['__dirname + '**/*.entity{.ts,.js}'],
};
```

Configurando banco de dados

O próximo passo será configurar a entidade users para que o TypeORM mapeie e crie o espelho da tabela no banco:

```
import { Module } from '@nestjs/common';
import { TypeOrmModule } from '@nestjs/typeorm';
import { AppController } from './app.controller';
import { AppService } from './app.service';
import { UsersModule } from './users/users.module';
import { config } from './ormconfig';

@Module({
  imports: [UsersModule, TypeOrmModule.forRoot(config)],
  controllers: [AppController],
  providers: [AppService],
})
export class AppModule {}
```

Configurando banco de dados

O próximo passo será configurar a entidade users para que o TypeORM mapeie e crie o espelho da tabela no banco:

```
import { Column, Entity, PrimaryGeneratedColumn } from 'typeorm';
```

```
@Entity()  
export class User {  
  @PrimaryGeneratedColumn('increment')  
  id: number;
```

```
  @Column()  
  nome: string;
```

```
  @Column()  
  idade: number;  
}
```

Configurando banco de dados

Veja que utilizamos o decorator `@Entity()` para apontar que a classe `User` é uma entidade, e suas propriedades sendo colunas, assim como o `id` sendo uma chave primária.

```
npm run start
```

Criando um usuário

Primeiramente vamos criar um usuário, para isto precisamos definir as propriedades no arquivo **create-user.dto**, neste caso, o id será gerado automaticamente, portanto vamos passar o nome e idade:

```
export class CreateUserDto {  
  nome: string;  
  idade: number;  
}
```

Criando um usuário

O próprio TypeORM nos fornece uma série de recursos que podemos utilizar em nossa aplicação, como injetar um repositório referente a entidade que estamos trabalhando na camada de service, como por exemplo no **users.service.ts**:

```
@Injectable()
export class UsersService {
  constructor(
    @InjectRepository(User)
    private userRepository: Repository<User>,
  ) {}
  create(createUserDto: CreateUserDto) {
    return this.userRepository.save(createUserDto);
  }
}
```

Criando um usuário

Para isso, utilizamos o **@InjectRepository(User)** e então criamos um objeto do tipo **Repository** passando no construtor da classe **UserService**, utilizando a injeção de dependências. Desta forma podemos chamar os métodos que irão se comunicar com o banco e efetivamente manipular os dados, neste primeiro exemplo estamos criando um usuário utilizando o método **create()**.

Criando um usuário

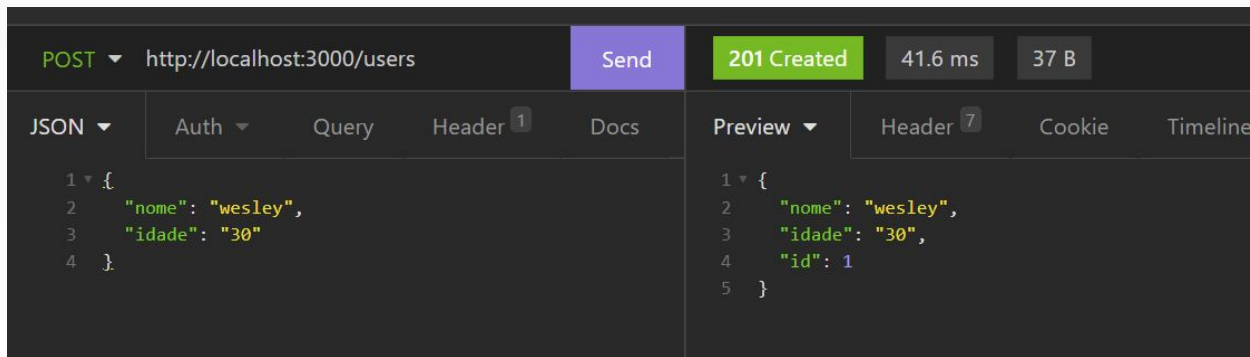
O controller é o responsável pelas rotas e pelas requests/responses, para criar um recurso vamos utilizar o verbo POST e chamar o método create do service, conforme abaixo:

```
@Controller('users')
export class UsersController {
  constructor(private readonly userService: UserService) {}

  @Post()
  create(@Body() createUserDto: CreateUserDto) {
    return this.userService.create(createUserDto);
  }
}
```

Criando um usuário

Agora é simples, no controller temos a rota users e a requisição POST, onde vamos receber um createUserDto, chamar o método create do userService para criar um novo registro.



The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** http://localhost:3000/users
- Status:** 201 Created
- Time:** 41.6 ms
- Size:** 37 B
- Request Body (JSON):**

```
1 {  
2   "nome": "wesley",  
3   "idade": "30"  
4 }
```
- Response Body (JSON):**

```
1 {  
2   "nome": "wesley",  
3   "idade": "30",  
4   "id": 1  
5 }
```

Criando lista e buscando usuário único

```
@Get()  
findAll() {  
  return this.userService.findAll();  
}
```

```
@Get('/:id')  
findOne(@Param('id') id: number) {  
  return this.userService.findOne(id);  
}
```

Criando lista e buscando usuário único

```
findAll() {  
  return this.userRepository.find();  
}
```

```
findOne(id: number) {  
  return this.userRepository.findOneBy({ id: id });  
}
```

Removendo um usuário

```
remove(id: number) {  
  return this.userRepository.delete(id);  
}
```

```
@Delete('/:id')  
remove(@Param('id') id: number) {  
  return this.userService.remove(id);  
}
```

Instalar Insomnia

<https://sempreupdate.com.br/instale-o-insomnia-no-ubuntu-fedora-debian-e-outras-distribuicoes-linux-com-flatpak/>

Questionário



Atividade

Dentro do seu projeto NestJS, crie:

- Rotas CRUD para o módulo Usuário;
- Services que atenda as suas rotas CRUD dentro do arquivo de pessoa.service;
- Dtos e entity de Usuário;